

* Distributed Database :

A distributed database is a collection of multiple, logically interrelated databases distributed over a computer network, managed by a distributed database management system (DDMS),

- Data is distributed across various servers in distributed database.
- The servers are present at various geographical locations and connected via network.
- The distribution is transparent to user. The system appears as a single database, even though data may be stored in different locations.

* Need for distributed database approach

- Organizations today often operate across multiple locations, serve large user bases, and handle huge volumes of data.
- A centralized database may become a bottleneck or single point of failure.
- The scale of data affects response time and down time.
- A distributed database system (DDBS) overcomes these limitations by storing and processing data across multiple interconnected sites.

1. Improved Reliability and Availability

- If one site fails, other can still function using replicated data.
- Reduces down time

2. Faster Local Access:

- Data Stored closer to the Site where it is used reduces access time and network load.

3. Scalability

- Easy to add new Site

4. Cost Efficiency:

- Uses cheaper, smaller computers at multiple Sites.

5. Data Sharing and Integration:

- Enables Sharing of data between multiple

6. Improved Performance:

- Parallel query processing and load distribution reduce response time.

* Types of Distributed Databases

a) Based on homogeneity

- Homogeneous
- Heterogeneous

b). Based on data distribution

- Replicated
- Fragmented
- Hybrid

* Homogeneous Distributed Database:

- A Homogeneous Distributed Database is a type of distributed database in which

all sites use the same DBMS software, data model, and schema.

- Because the software environment is identical, the system appears as a single unified database to the user.

- Characteristics :

- Same DBMS Software : All sites run the same database system (eg MySQL etc),

- Same Data Model - Uniform relational, document or object-oriented model.

- Same Query Language - Identical SQL dialect at all sites.

- Common Schema - Same structure, table names, and relationships.

- Easy integration - Minimal data conversion or transformation needed.

★ Heterogeneous Distributed Database:

- A Heterogeneous Distributed Database is a type of distributed database in which different sites may use different DBMS software, data models, query languages, and schemas.

- The system integrates these diverse databases so that users can access them as if they were a single unified database.

- Middleware Requirement : Needs special software to enable communication and data conversion between sites.

* Replicated Distributed Database:

- Replicas (Copies) of the same data is stored at multiple sites.
- It improves data availability, reliability and performance for local queries.
- Replication can be of two types:
 - Full Replication
 - Partial Replication
- It has high fault tolerance.
- It has high storage and update cost.

* Fragmented Distributed Database:

- In this the database is divided into smaller logical units (fragments), and each fragment is stored at a different site.
- The purpose is to store data closer to where it is most frequently accessed, reducing communication cost.

1) Horizontal Fragmentation:

Horizontal fragmentation splits a table by rows based on conditions on one or more attributes.

- Each fragment contains a subset of rows, which are then stored at different sites.

- Each fragment has same columns as the original table but different rows.

ex: A customers table is divided into customers_Mumbai and customers_Delhi based on City.

2) Vertical Fragmentation :

- Divides a table into subsets of columns.
- Each fragment contains a subset of attributes and must include a primary key to link with other fragments.

Ex. A Students table is split into

Student_Personal (name, address, roll_no) and
Student_Academics (roll_no, marks)

3) Hybrid Fragmentation :

Combination of horizontal and vertical fragmentation.

ex: First horizontally divide by city, then
vertically divide attributes.

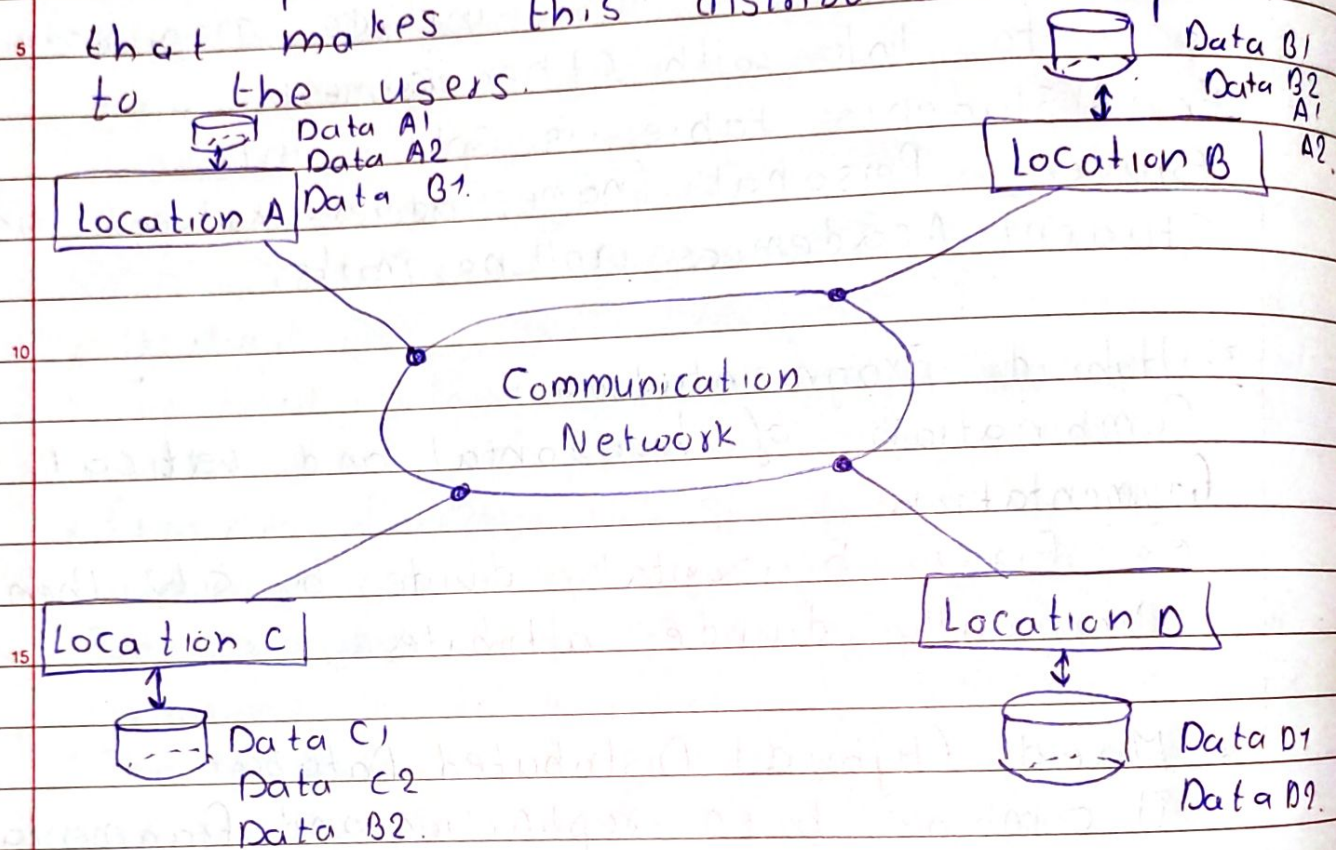
* Mixed (Hybrid) Distributed Database.

- It combines both replication and fragmentation.
- Example - Some data is fragmented, and each fragment is replicated.
- It provides high fault tolerance and availability along with good performance.

Which Systems are not considered as distributed Systems.

- Multiprocessor System
- Tightly coupled System or loosely coupled multiprocessor
- Time Sharing Computer Systems.
- Isolated databases.
- Batch Processing Systems.
- db residing on one of the nodes of a network.

A distributed database management system is the software that manages the DDB and provides an access mechanism that makes this distribution transparent to the users.



* Transparency:

- The general idea of hiding implementation details from end users is called transparency.
- The system appears like a single, integrated database to the users even though the data is actually stored across multiple sites.
- The fundamental issue is to provide data independence in distributed environments.

- Network (distribution) transparency
- Replication transparency
- Fragmentation transparency

* Types of transparency:

1. Network Transparency:

- Users should not need to know the network structure, location of resources, or how communication is done across sites.
- ex. Select * from Students where RollNo=101; Should work without the user knowing whether Students table is in Mumbai, Delhi or split across both.
- It hides communication protocols, network addresses and paths.

2. Fragmentation Transparency:

- Hides the fact that a relation (table) is divided into fragments and possibly stored at different sites.
- Types →
 - Horizontal Fragmentation (Selection)
 - • Vertical Fragmentation (Projection)
 - • Hybrid.
- Ex even if student table is split into Student_personal and Student_marks a query "Select * from Students" must return user result similar to single database.

3. Replication Transparency:

- It hides the fact that replicas (copies) of

data exist at multiple sites for fault tolerance and performance.

Ex: If order table is replicated at Site A and Site B, and Site A is down, the query should still work using Site B. Without user being aware of the switch.

10 * **Scalability:**
Scalability is the ability of a distributed database system to handle increasing amounts of data, users, or transactions by adding resources such as more nodes or storage, without significant degradation in performance.

1. Horizontal Scaling (Scale-Out)

• Increasing capacity by adding more machines or nodes to the distributed system.

2. Vertical Scaling (Scale-Up)

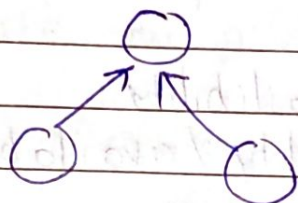
• Increasing capacity by upgrading the hardware of existing nodes.

25 * **Parallelism:**

Parallelism in distributed databases refers to the ability to execute multiple operations simultaneously across different sites or processors to improve system performance, throughput, and reduce response time.

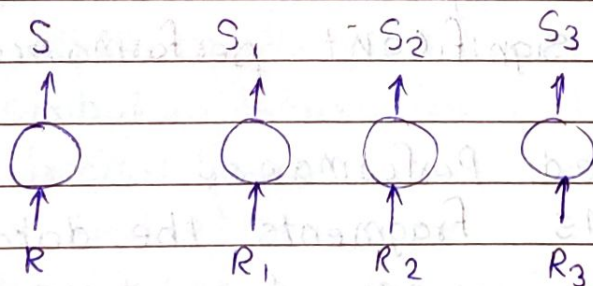
• Inter-Query Parallelism:

- Different queries are executed in parallel on different nodes.
- Improves throughput by handling multiple users simultaneously.



• Intra-query Parallelism:

- A single query is divided into subqueries that run in parallel.
- Speeds up execution time for complex queries.



★ Distributed DBMS promises:

It refers to the guarantees or features that the DDBMS aims to provide to users.

1. Transparency:

- Transparent management of distributed, fragmented and replicated data

• The DDBMS tries to hide the complexities of distribution from the user

• Different kind of transparency include

- Location transparency
- Replication transparency
- Fragmentation transparency
- Concurrency transparency
- Failure transparency

2. Reliability and Availability:

- Improved reliability/availability through distributed transactions.
- DDBMS promises to maintain data availability even if some nodes fail, often by replicating data or rerouting queries.

3. Scalability:

- The system can grow by adding more nodes, handling more data and users without significant performance loss.

4. Improved Performance:

- A DDBMS fragments the database by keeping the data closer to where it is needed most.
- Data localization reduces the contention for CPU and I/O services and simultaneously reduces access delays involved in wide area networks.

5. Easier Expansion:

- Same as Scalability, adding more processors is much easier.

6. Parallelism:

- Interquery and Intraquery parallelism can be achieved by executing multiple queries at different sites, or by breaking up a query into a number of subqueries that execute in parallel at different sites.

7. Concurrency Control:

- System manages simultaneous operations on data without conflicts or corruption.

8. Autonomy:

- Each site in distributed system can operate independently to some extent, yet participate in global operations.

* Distributed DBMS Issues:

1. Data Distribution Issue

- Data Fragmentation
- Data Replication:
- Data Allocation

2. Distributed database design:

- How to distribute the database.

3. Data Integrity and Consistency

- Ensuring ACID properties.

4. Distributed Query processing:

- Optimization problem.
 $\min\{cost\} = \text{data transmission} + \text{local processing}$
- Query optimization is harder in distributed setting.
- General formula is NP-Hard.

5 Distributed Concurrency Control I.
Synchronization of concurrent accesses

- 6. Overall Cost:
- 7. Complex Nature
- 8. Security.

* Functional Layer of Centralised Database System:

- This layer handles the logical processing and control of database operations.
- It ensures data integrity, query processing, and transaction management within one system.

• It includes

1. Query Processor - Parses SQL query.
2. Transaction Manager - Manages ACID.
3. Concurrency Control Manager - Concurrency
4. Recovery Manager - After crashes recovery.

1. Query Processing.

- Parses user queries
- Translates high level queries into low level operations.
- Optimizes query execution plans to improve performance.

2. Transaction Management:

- It ensures that database transactions comply with ACID properties.
- Manages transaction states.

3. Concurrency Control

- Manages Simultaneous Operations to prevent Conflicts.
- Prevent issues like lost updates, dirty reads, or deadlocks.

4. Recovery Management

- Ensures the System can recover from crashes or failures.
- Maintains logs and checkpoints.
- Supports rollback and recovery to a consistent state.

* Execution Layer in a Centralized Database

- The execution layer is the lowest operational layer in the DBMS architecture.
- It is responsible for physically performing database operations as per the instructions received from the higher layers (such as functional layer).

• The execution layer is the physical operation layer of a centralized DBMS that executes the query execution plan, accesses the actual stored data, manages low-level I/O operations and ensure results are returned correctly to the user or application.

* Main Responsibilities

1. Accessing Physical Storage

- Read and writes data block from the database

files stored on disk.

- Uses buffer management to minimize disk I/O.

2. Index and Record Management.

- Navigates through indexes to locate records quickly.

3. Interfacing with Hardware.

- Performs actual disk I/O, memory access, and CPU-level operations.

4. Transaction Logging and Recovery.

- Maintains transaction logs for rollback and crash recovery.

5. Following Execution Plans.

- Implements the optimized query execution plan generated by the functional layer.

* Distributed Database Architectures.

1. Client-Server Architecture.

- It is one of the most widely used architectural models for distributed databases.

- It divides the system into two main components:

- Client: The frontend component that request services.

• Overall better System price/performance

camlin

Date : _____

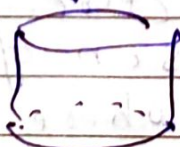
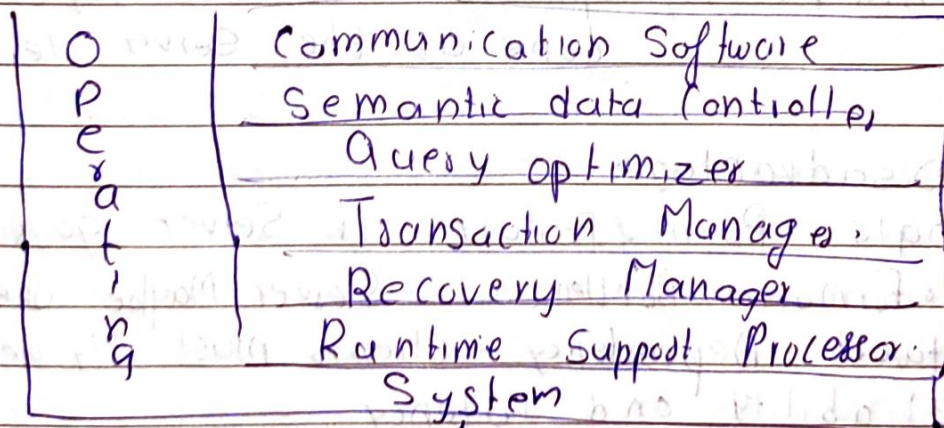
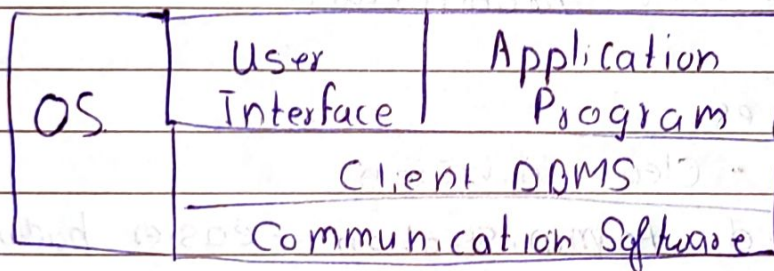
• Server - backend components that provide services.

• The Clients and Servers communicate over a network and follow a request-response model.

* Logical Separation of Concerns.

• Client handle the presentation and user interface logic

• Servers handle data processing and database management.



Database

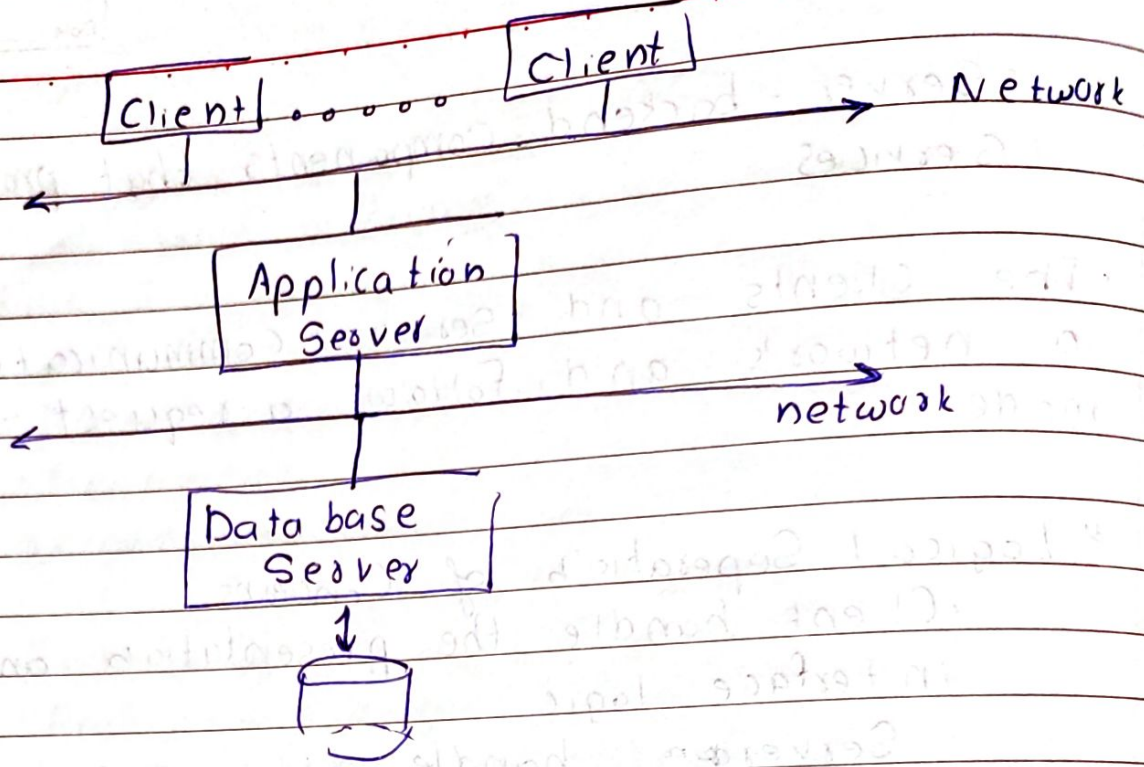


Fig. 3-Tier Architecture.

* Advantages.

- Simplicity - Clear division
- Centralized Administration - easier backup, recovery
- Scalability - You can add more clients.
- Maintainability - Updates to database logic are done at the Server level.

* Disadvantages.

- Single Point of Failure - If Server goes down...
- Performance Bottlenecks - Server Maybe Overloaded
- Network Dependency - Clients must rely on network reliability and latency.

2

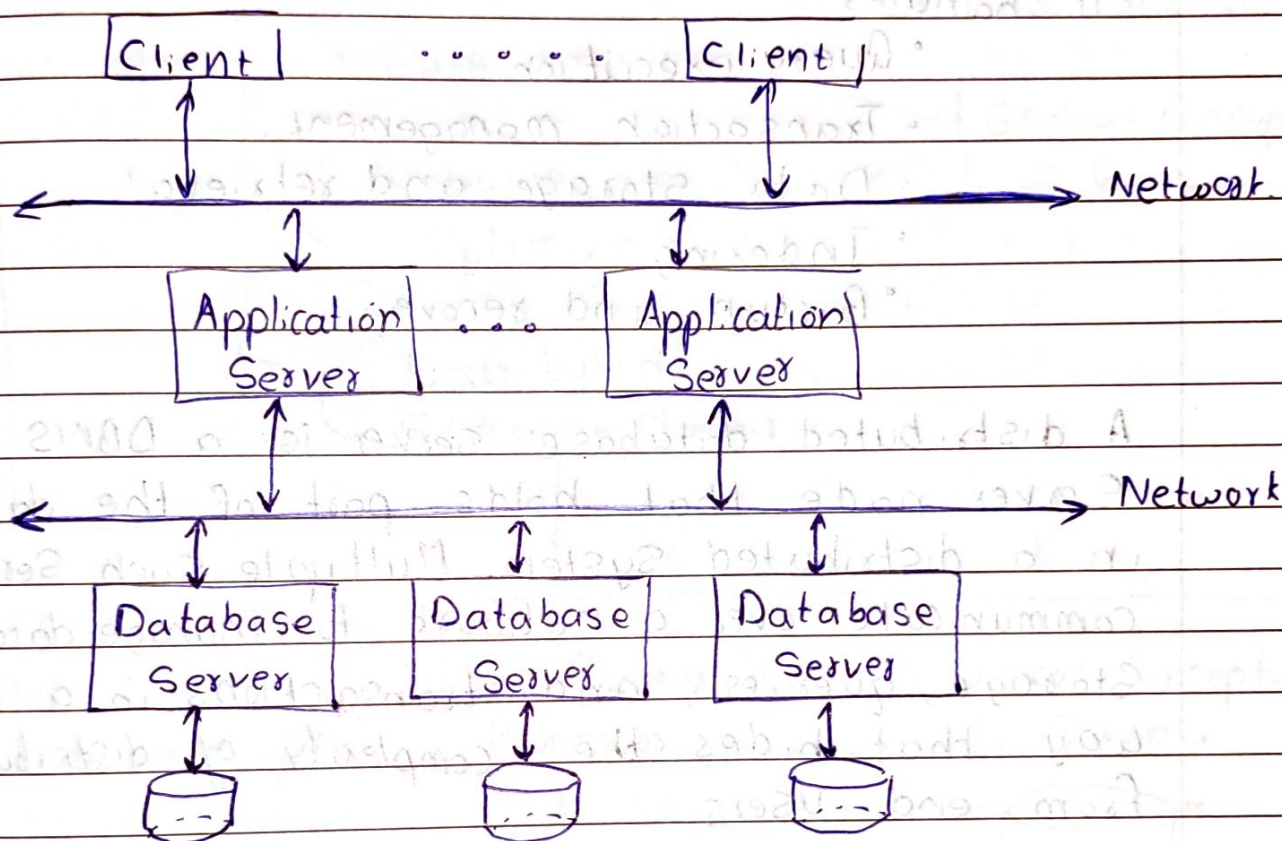
3 - Tier Architecture.

- It is an extension of the client-server model, where an additional middle tier sits between the client and the database.

Layers in 3-Tier Architecture.

a) Presentation Tier (Client level)

* Distributed Database Server



Application Server

- The application Server acts as the middle layer between Clients and database Servers.
- Business logic is implemented here.
- It accepts request from clients → break them into sub-queries → Send to correct database Server.
- It then integrates and processes results before sending them back to the client.
- It improves Security by hiding direct database access.
- It reduces load on clients by handling heavy computation.

* Database Server:

This is the back-end layer where actual data storage and management happens.

• This Server runs on some DBMS like MySQL, PostgreSQL etc.

• It handles.

- Query execution

- Transaction management.

- Data storage and retrieval

- Indexing

- Backup and recovery.

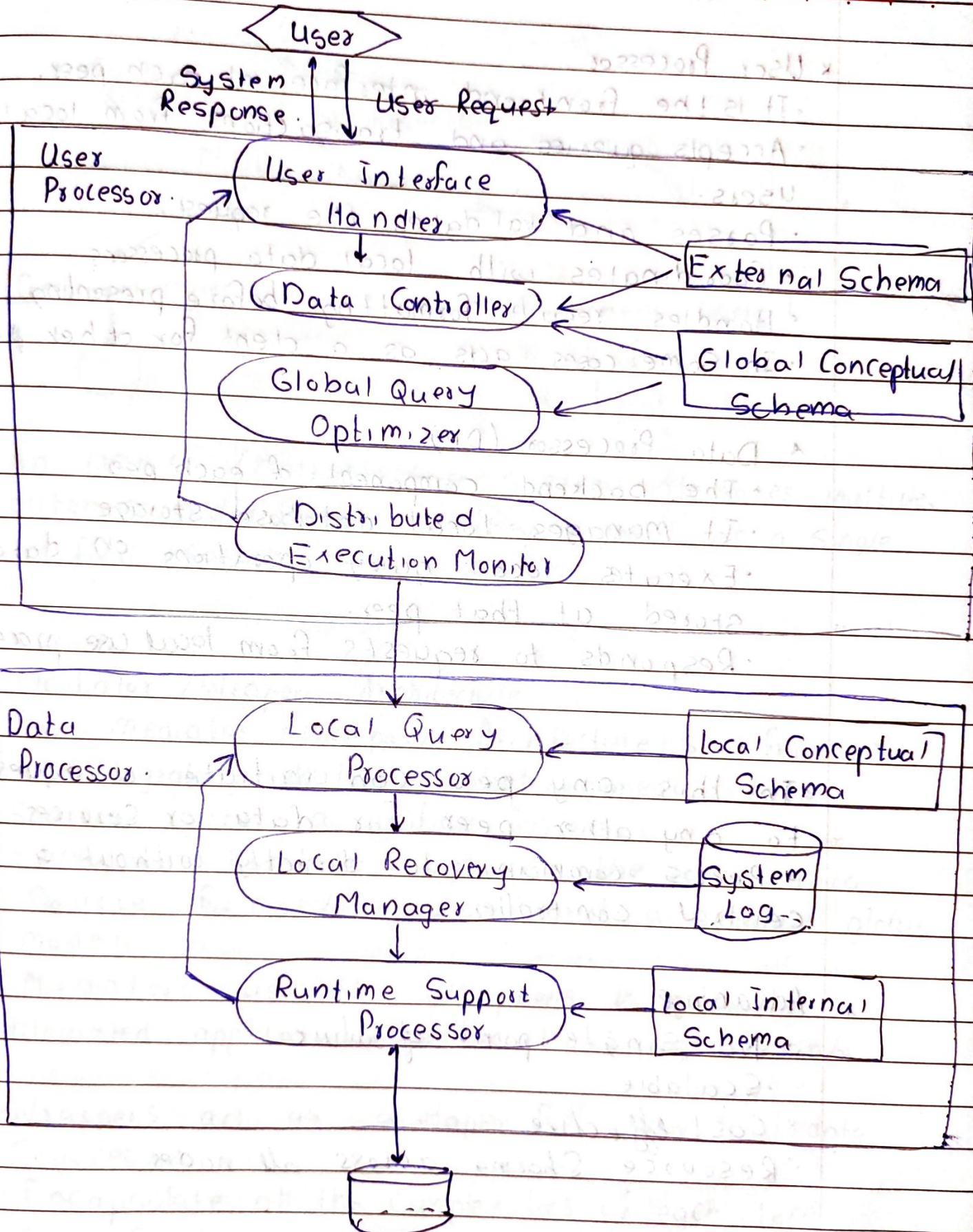
A distributed database server is a DBMS server node that holds part of the database in a distributed system. Multiple such servers communicate over a network to manage data storage, queries, and transactions in a way that hides the complexity of distribution from end users.

* Peer-to-Peer Component Architecture.

Peer-to-Peer (P2P) architecture is a distributed system model in which every participating node (peer) has equal status and can function both as client and a server.

- There is no dedicated central server.

- Each peer can initiate or respond to requests for service and data.



* User Processor :

- It is the front-end interface at each peer.
- Accepts queries and transactions from local users.
- Passes and validates the request.
- Coordinates with local data processors.
- Handles result formatting before presenting.
- In some cases, acts as a client for other peers.

* Data Processor (DP)

- The backend component of each peer.
- It manages local database storage.
- Executes local query operations on data stored at that peer.
- Responds to requests from local user processors.

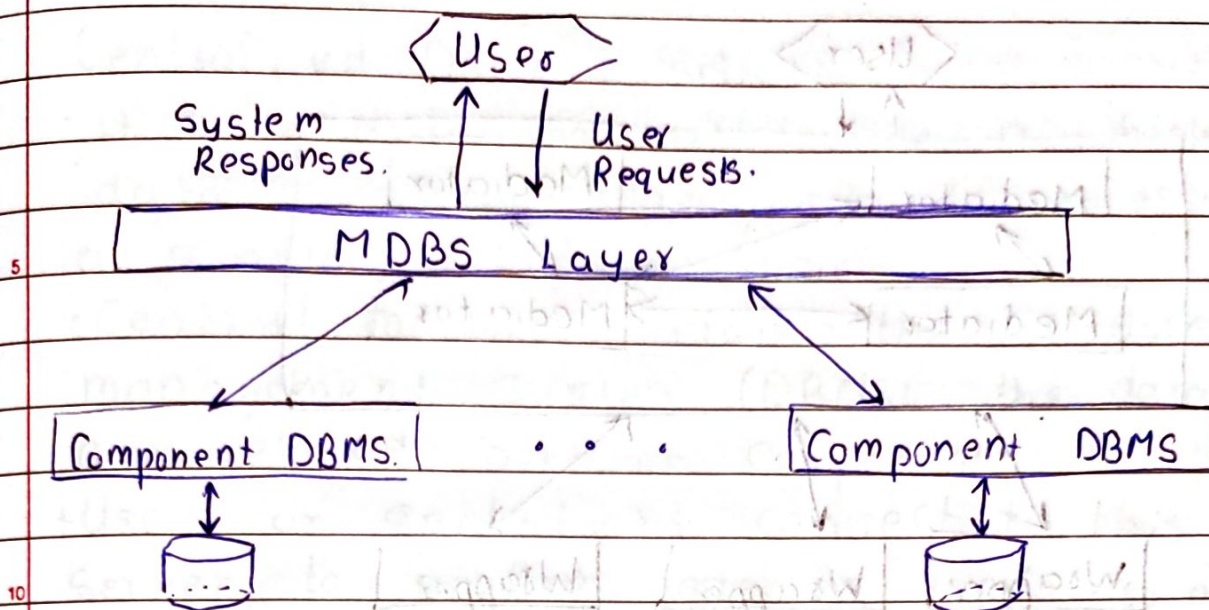
• In this any peer can initiate a request to any other peer for data or services.

- Peers communicate directly without a central controller.

* Advantage

- No single point of failure
- Scalable
- Cost effective
- Resource sharing across all nodes

* MDBS Components & Execution.

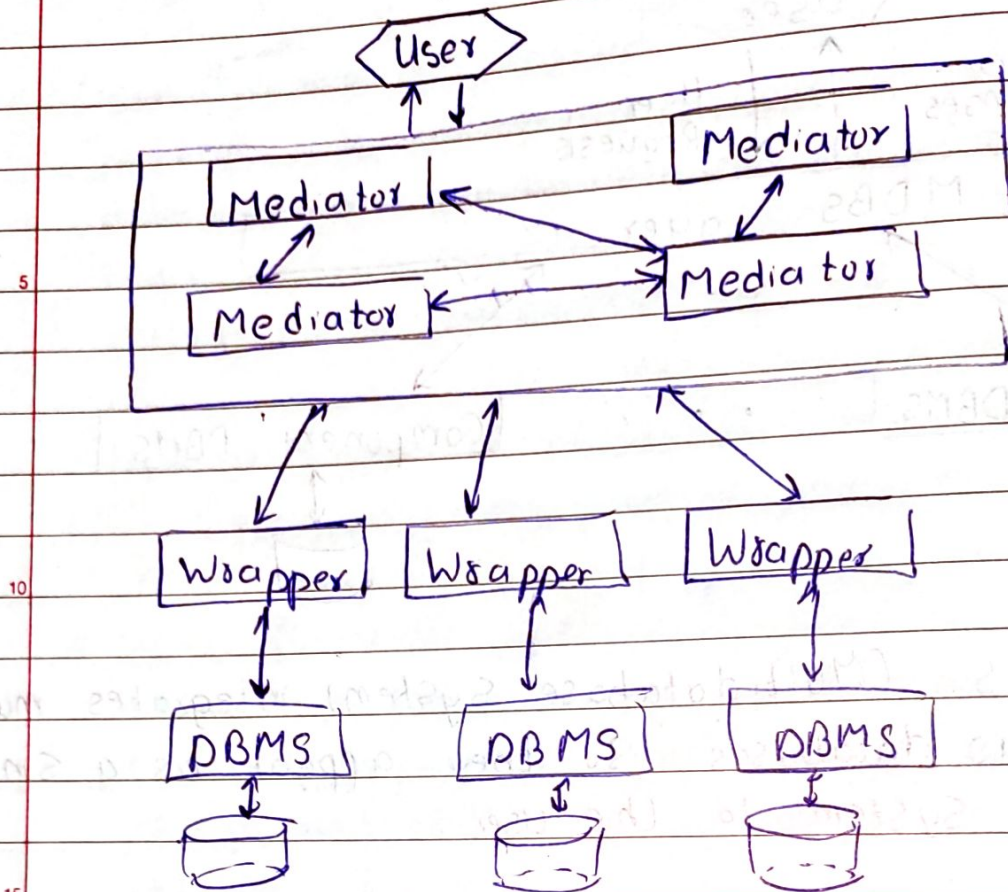


An MDBS (Multidatabase System) integrates multiple autonomous databases so they appear as a single unified system to the user.

* Mediator / Wrapper Architecture.

The mediator / wrapper architecture is a framework for integrating multiple heterogeneous data sources into a unified system.

- Wrappers translate between the local data source format/language and a common global model.
- Mediators use the wrappers to provide integrated, application-specific views of the data.
- Wrappers act as an adapter for individual data sources.
- Encapsulate all the complexities of each local data source.
- Each data source has its own wrapper.



- Mediators Sit above the wrappers.
- They receive queries from users or applications.
- Decompose the query into sub-queries for relevant wrappers.
- Combines results from multiple wrappers into a single unified answer.

* Advantages.

- Hides heterogeneity - Users see only a single interface.
- Separation of concerns - mediators focus on integration, wrappers focus on translation.

* Database System Architectures.

1. Centralized Database Systems

- It is a database architecture in which all data is stored, managed, and processed on a single computer system.
- Central machine contains the database management system (DBMS), the database files, and related software components.
- Users or applications connect to this central server to perform queries, updates, and other operations.

* Characteristics.

- Runs on a single computer system.
- Single-user systems (Embedded databases).
- Multi-user systems (Server systems): Client sends request to server via a network.
- Parallel processing: Multicore systems allow coarse-grained parallelism.

* Advantages

- Centralized control → Simplified management.
- Consistency → All users access same up-to-date data.
- Efficiency → Optimized for large transaction handling.

* Disadvantages

- Single point of failure
- Scalability limits

2. Server System Architecture
Server Systems in database architectures can be broadly classified into two types:

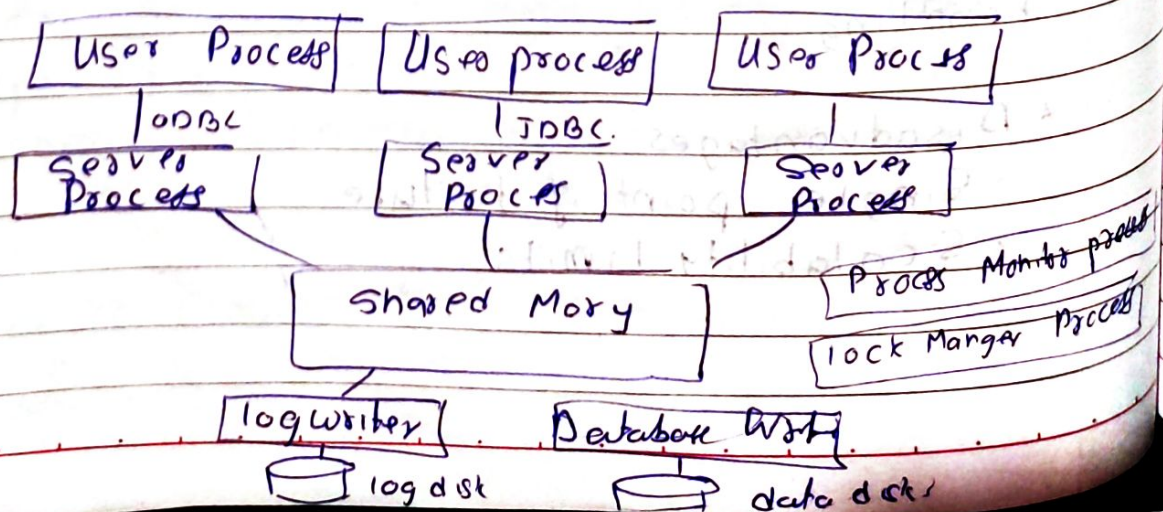
7. Transaction Servers:

- These servers handle transaction requests from multiple clients.
- These are also called query servers or SQL Server Systems.
- Clients send SQL queries or transaction commands to the server, and the server executes them and returns the results.

Working:

1. A client sends a request (eg. SQL statement).
2. The server processes the request, performs the required transaction logic, and accesses the database.
3. The result is sent back to the client.

- Communication is done through RPC.
- Applications typically use ODBC/JDBC APIs to communicate with transaction servers.



• Shared memory contains: Shared data.

• Buffer pool

• Lock table

• Log buffer.

• Cached query plans.

• To ensure that no two processes are accessing the same data structure at the same time, databases systems implement mutual exclusion using either

• Atomic instructions

• Test-And-Set

• Compare-And-Swap.

• Operating System Semaphores.

* Test-and-Set:

• It is a hardware-supported atomic instruction used in multiprocessor and multithreaded environments to implement synchronization primitives like locks.

• It prevents race conditions by ensuring that checking and modifying a shared variable happens as an indivisible (atomic) operation.

• The test and set happen together, without any other process or thread being able to interrupt in between.

Algorithm. Test-And-Set(M)

- Memory location M, initially 0
- Test-and-Set (M) sets M to 1, and returns old value of M.

• It returns 0 to indicate process has acquired the mutex

• Return value 1 indicates Someone is already holding the mutex

• Must try later.

• To release mutex M is Set again to 0.

• It is usually implemented at the machine instruction level in CPU hardware.

• It uses memory locking to ensure no other CPU accesses the variable during the instruction.

* Compare and Swap:

• It is a hardware-supported atomic instruction used to achieve Synchronization without using heavy locking mechanisms

• The purpose is to update a shared variable only if it has not been changed by another thread since it was last read preventing race conditions.

• The comparison and the Swap happen as one indivisible operation.

Working:

The CAS instruction:

- Takes three arguments:
 - address $\rightarrow$ Memory location to check and Update
 - expected $\rightarrow$ Value you expect at location
 - new-value $\rightarrow$ The value to write if the
- Compares the value in memory with expected
- If equal $\rightarrow$ replaces it with new-value.
- If not equal $\rightarrow$ does nothing.
- Returns the original value found in memory.

Algorithm

Compare-and-Swap ($M, V1, V2$)

- Atomically do following
 - If $M = V1$, Set $M = V2$ and return success
 - Else return failure.
- With $M = 0$ initially, $CAS(M, 0, 1)$ equivalent to test-and-set(M)
- Can use $CAS(M, 0, id)$ where $id = \text{thread-id}$ or process-id to record who has the mutex.

2. Data Servers

- These servers focus on high-performance data access and parallel transaction execution.
- Instead of processing everything in one central server, the data is partitioned and processed in parallel across multiple processors or nodes.
- It allows faster performance using parallelism

- Prefetching : Prefetch items that may be used Soon.

- Data Caching

- Cache coherence (Storing Copies of data locally)

- Lock Caching

- Locks can be cached by client across transactions.

- Adaptive lock granularity.

- Lock granularity escalation : Switch from fine granularity lock to coarser.

- Lock granularity de-escalation : Start with coarse granularity to reduce overheads, switch to finer granularity in case of more concurrency conflict at server.

- Data Caching

- Data can be cached at client even in between transactions.

- But we need to check if data is up-to-date before using (Cache Coherency)

★ Parallel Systems

A parallel database system uses multiple processors and multiple disks that are connected via a high speed interconnection network.

• The goal is to handle workloads that exceed the capacity of a single computer system by dividing tasks among multiple processors.

* Types of Parallel Machines.

* Coarse Grain Parallel Machine:

- Small number of powerful processors.
- Suitable for medium-to-large workloads where each processor can handle substantial processing power.

* Massively Parallel (Fine-Grain) Machine.

- Thousands of smaller processors working together.
- Typically deployed in data centers.
- Well-suited for extremely large workloads where each processor can handle substantial processing.

* Key Performance Measures.

1. Throughput.

- Number of tasks completed in a given time interval.
- Higher throughput means the system can handle more work simultaneously.

2. Response Time

- Time taken to complete a single task from submission to completion.
- Lower response time means faster individual task execution.

* **Speed Up:** how much faster a Speedup measures how much faster a fixed-size problem can be executed on a larger system compared to a smaller one.

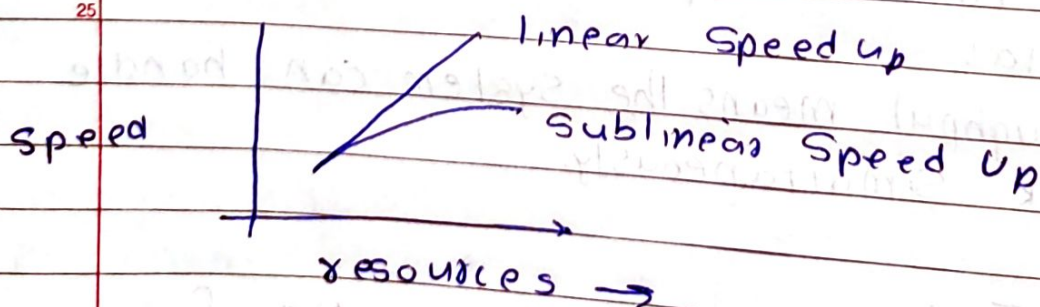
$$\text{Speedup} = \frac{\text{Elapsed time on Small System}}{\text{Elapsed time on larger system}}$$

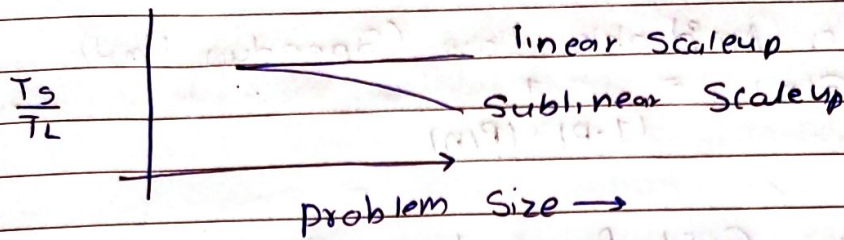
• If you give an N -times larger system to the same problem, the ideal linear Speedup is N .

* **Scale Up:** Scaleup measures the ability of a system to handle a proportionally larger problem as the system size increases.

$$\text{Scaleup} = \frac{\text{Small System Small problem elapsed time}}{\text{big system big problem elapsed time}}$$

• Scaleup is linear if equation equals 1.





* Batch Scaleup :

Focuses on handling single large job when both problem size and system size grows.

- Use N -times larger computer on N -times larger problem.

* Transaction Scaleup :

It focuses on many small independent queries being processed in parallel when both number of users and system / database size grows.

- N -times as many users submitting requests to N -times larger database, on an N -times larger computer.
- Well suited for parallel execution.

* Factors limiting Speedup and Scaleup.

Speedup and Scaleup are often sublinear due to :

- Even in parallel systems, some work must remain sequential.
- There's also overhead for starting, coordinating and synchronizing processes.
- Startup Cost: Starting up multiple processes

• As per Amdahl's Law (Speedup limit)

$$S(n) = \frac{1}{(1-p) + (P/n)}$$

• As per Gustafson's law: Scaleup limit

$$\frac{1}{n(1-p) + P}$$

• Interference:

When multiple processes compete for shared resources they interfere with each other.

• This adds wait time

• Skew: (load Imbalance)

• Increasing degree of parallelism increases variance.

Some process finish early while other lag behind.

• Overall execution time is determined by slowest of parallelly executing tasks

* Interconnection Architectures

1. Bus:

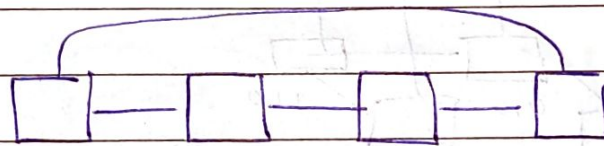
All processors share a common communication medium



- Simple and inexpensive
- Easy to add/remove processors,
- But, only one transaction at a time $\rightarrow$ contention
- Doesn't Scale well

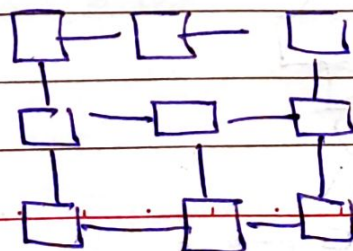
2. Ring :

- Processors are connected in a circular fashion, each node connects to exactly two neighbors.
- Simple layout
- Every processor has equal access.
- Data may need to travel across multiple nodes $\rightarrow$ higher latency.
- Failure of one node can break ring



3. Mesh

- Processors are arranged in a 2D grid, each connected to neighbors.
- Good Scalability
- Local communication is fast
- Many parallel paths $\rightarrow$ reduces contention
- Edge processors have fewer connections $\rightarrow$ uneven performance.
- High communication delay if far apart nodes must communicate.



4. Hyper Cube:
Processors are connected as vertices of a

d-dimensional cube.

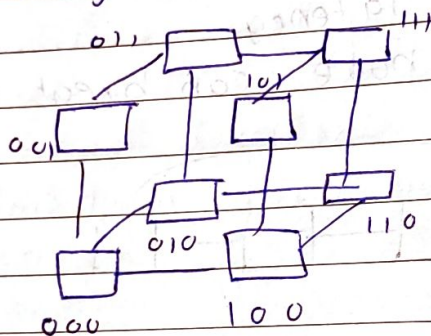
Each processor has d neighbors
(where $d = \log_2 N$ for N processors).

- High Connectivity. → Short paths
- Uses logarithmic distance.
- Scales well as N grows.

• More complex

• Difficult to implement physically

• Each processor are marked by binary
and neighbors differ by 1 unit.



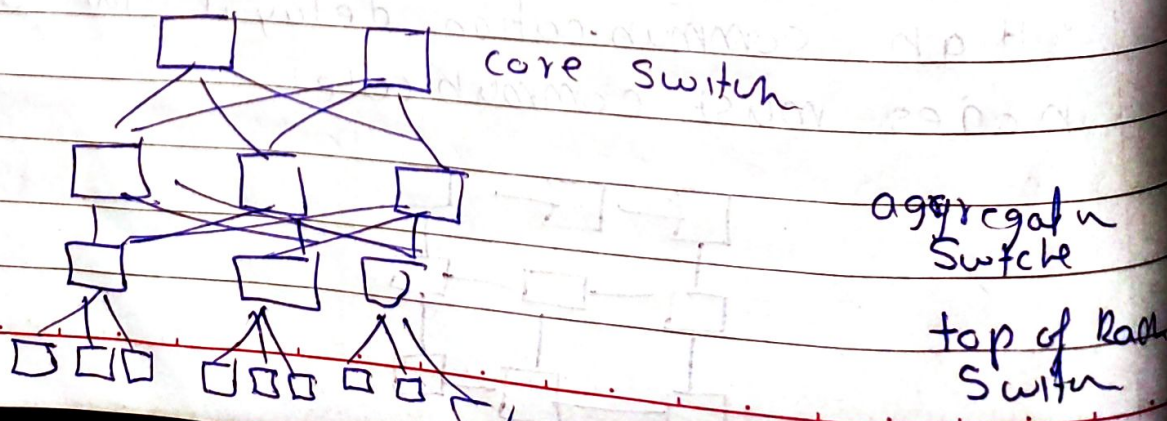
5. Fat Tree Topology,

• Widely used in data centers

• Top of rack Switch has 40 machines

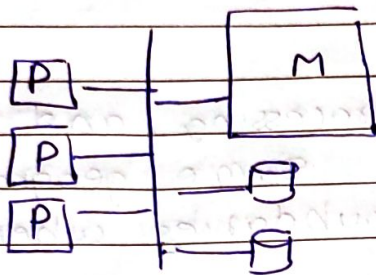
• Each TOR Switch are connected to multiple aggregation Switches.

• Aggregation Switches connect to multiple core Switches.

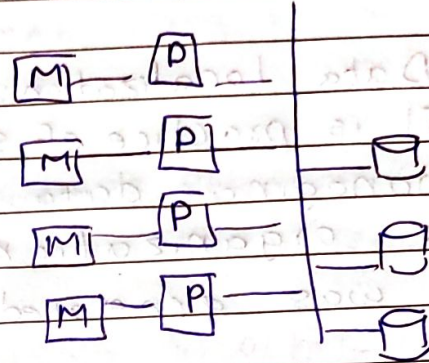


- It provides equal cost multipaths.
- Higher fault tolerant.
- Scales well.

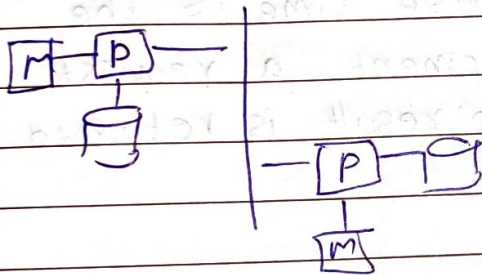
* Parallel Database Architecture



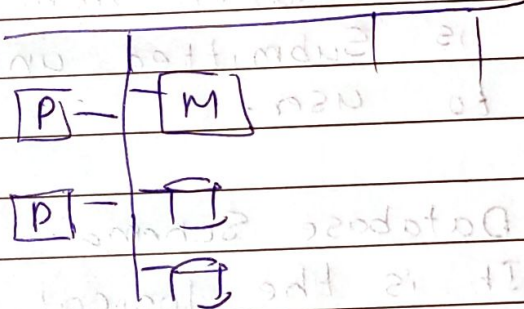
Shared Memory.



Shared disk.



Shared Nothing.



hierarchical

* Fine Grain Parallelism.
It refers to dividing a task into very small sub-tasks (instructions or data operations) that can be executed in parallel.

* Data localization:
It is practice of storing processing, and managing data within some geographic or organizational boundaries where it was generated.

* Response time wrt parallel systems.
In parallel system, response time is the time taken from the moment a request is submitted until the result is returned to user.

* Database Schema.

It is the logical structure or blueprint of a database.

It defines how data is organized and how relationships between data are maintained.

* Function of Execution layer.

• It is responsible for executing query plan by accessing the database storage, applying operations, and returning to the user.

Select E.ENO, E.ENAGE From EMP E Join AGE A on
E.ENO = A.ENO where A.DUR > 24;

camlin

Date : _____

Begin Transaction,

IF exists (

Select 1

From Train

where TNO = 'T123' and Date = '2025-08-25',

AND STSOLD < CAP

)

Begin

Insert into PP (TNO, DATE, PNAME, CLASS, SPECIAL)

Values ('T123', '2025-08-25', 'ABC', 'Sleeps', 'None');

Update train Set STSOLD = STSOLD + 1,

where TNO = 'T123' and DATE = '2025-08-25',

commit;

end

else

Begin.

Rollback;

end;

$$S(n) = \frac{1}{f + \frac{(1-f)}{N}} = \frac{1}{(p-1) + \frac{(p)}{N}}$$

$$= \frac{1}{0.3 + \frac{0.7}{15}} = 2.88.$$

* Amdahl's Law.

let original total time $T_{\text{serial}} = 1$.

Now split the work into Serial and Parallel.

• Serial part takes $(1-p)$ fraction of time, cannot be speed up.

• Parallel part takes fraction p of the time, can be spread across N processors.

$$\Rightarrow T(N) = (1-p) + \frac{p}{N}$$

Speedup is ratio of old time to new time

$$S(N) = \frac{T_{\text{serial}}}{T(N)} = \frac{1}{(1-p) + \frac{p}{N}}$$

* Data Delivery Alternatives:

When designing a distributed system or information system, the delivery of data to users or applications can vary based on mode, frequency, and communication method.

1. Delivery Modes:

➤ ~~Push~~ Only. It refers to how data flows from sender to receiver in a distributed system or communication setup.

1. Pull-Only Mode.

- The receiver (client) requests data explicitly whenever it is needed.

- The sender does not send anything unless it receives a request.

ex. HTTP request, database queries.

- It is efficient when data is needed infrequently.

- Higher latency. (Since request must precede delivery).

2. Push-Only Mode

- The sender (Server) sends data to the receiver without waiting for any request.

- Delivery is proactive and continuous.

ex. Real time data, Stock prices, Sensor data.

- Very low latency.

- Good for real-time applications.

- It may overload receiver with unnecessary data.

3. Hybrid Mode.

Combines push and pull approaches. Some data is pushed automatically, while detailed or specific data is pulled by request.

- ex. Email notifications are push, attachments or old emails are pulled when user requests them.

- Balances efficiency and control.

- Reduces unnecessary push.

* Frequency of Delivery,

1. Periodic
• Data is delivered at regular, fixed interval regardless of changes.
• ex. Sensor reading temp every 5 mins
• It may send redundant data if values don't change.

2. Conditional (Event Driven)
• Data is sent only if certain conditions or thresholds are met.
• ex. CPU alert on usage exceeding 80%
• It requires monitoring and condition evaluation

3. Ad-hoc or Irregular:
• Data is delivered on demand or in irregular pattern
• ex. On demand video streaming
• It is highly flexible

* Communication Methods.

1. Unicast (One-to-One)

- Data is sent from one Sender to exactly one receiver
- ex. Client Server interaction
- It is not scalable.

2. One-to-many (Multicast/Broadcast)

- Data is sent from one Sender to multiple receivers simultaneously.

ex. live video broadcast

- Harder to manage delivery guarantees and reliability.